

بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department



CPCS-214 Computer Architecture and Organization

31 - Constructing ALU

Introduction

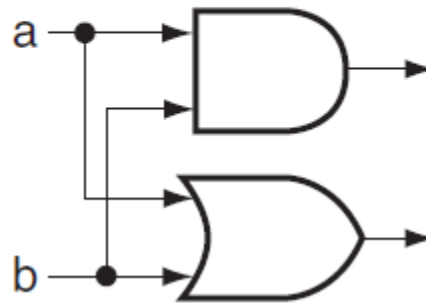
- Arithmetic logic unit (ALU) is the brawn of the computer
- Device that performs
 - Arithmetic operations: addition and subtraction or
 - Logical operations: AND and OR
- Will construct ALU from four hardware building block
 - AND and OR gates, inverters, and multiplexors

1-bit ALU

- Because the MIPS word is 32 bits wide, we need a 32-bit-wide ALU
- Start by constructing a 1-bit ALU
- Then connect 32 1-bit ALUs to create the desired ALU

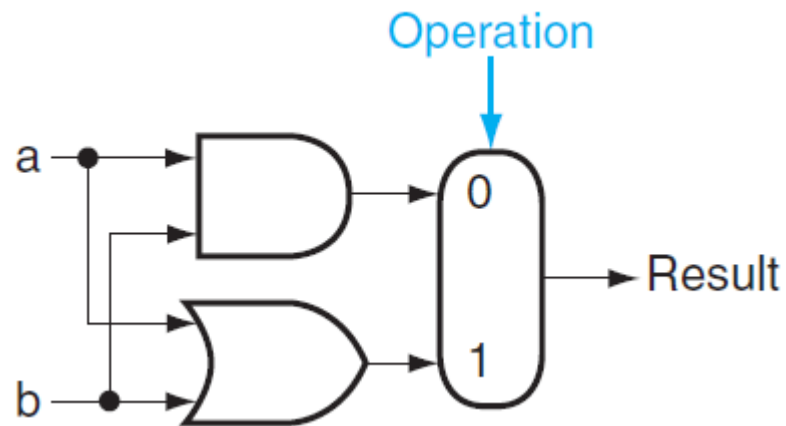
1-bit Logical Unit for AND, OR

- 1-bit logical unit for AND and OR is shown in figure



1-bit Logical Unit for AND, OR

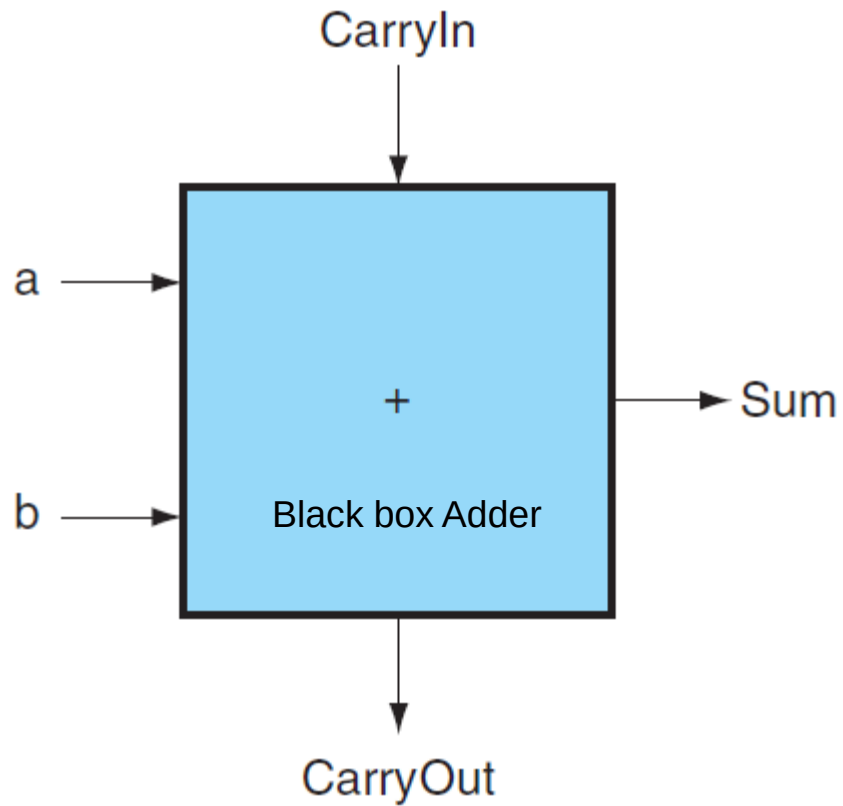
- 1-bit logical unit for AND and OR is shown in figure
- Multiplexor selects *a AND b* or *a OR b*
 - depending on whether value of *Operation* is 0 or 1



Addition

- Next function to include is addition
- Adder must have
 - two inputs for operands
 - single-bit output for sum
 - second output to pass on carry, called *CarryOut*
- Since *CarryOut* from the neighbor adder must be included as input, we need a third input
 - Called *CarryIn*

1-bit Adder

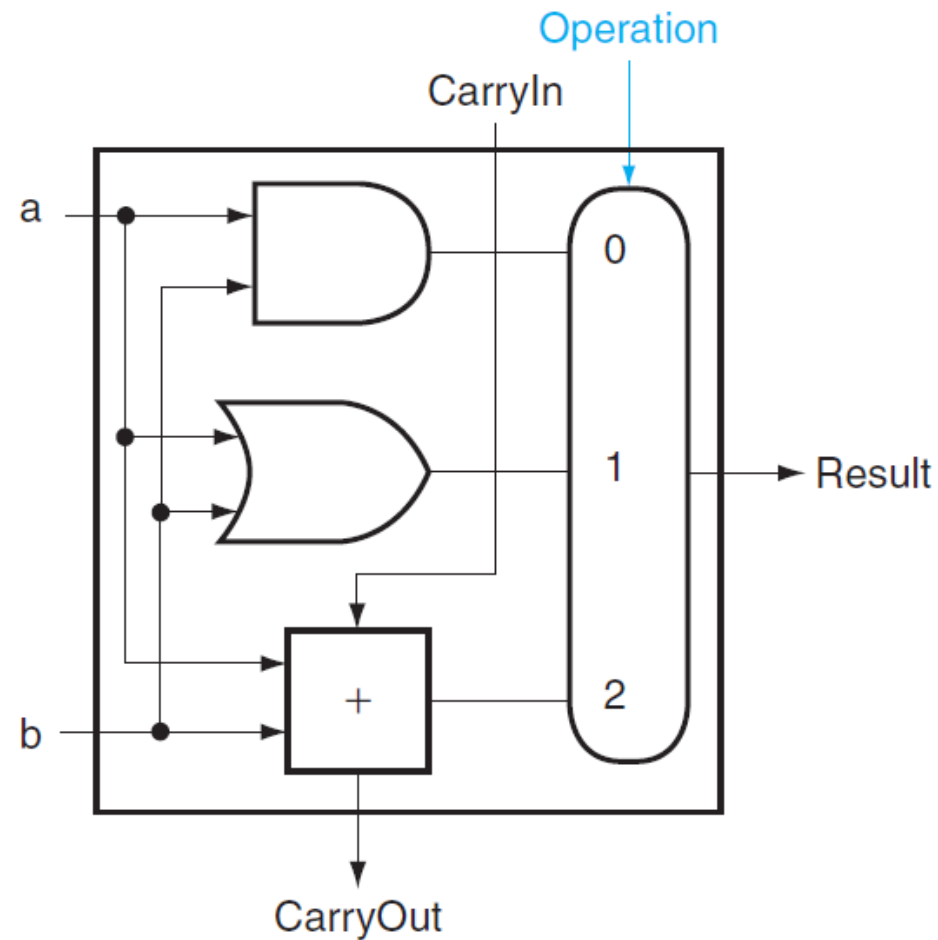


Inputs			Outputs		Comments
a	b	CarryIn	CarryOut	Sum	
0	0	0	0	0	$0 + 0 + 0 = 00_{\text{two}}$
0	0	1	0	1	$0 + 0 + 1 = 01_{\text{two}}$
0	1	0	0	1	$0 + 1 + 0 = 01_{\text{two}}$
0	1	1	1	0	$0 + 1 + 1 = 10_{\text{two}}$
1	0	0	0	1	$1 + 0 + 0 = 01_{\text{two}}$
1	0	1	1	0	$1 + 0 + 1 = 10_{\text{two}}$
1	1	0	1	0	$1 + 1 + 0 = 10_{\text{two}}$
1	1	1	1	1	$1 + 1 + 1 = 11_{\text{two}}$

A 1-bit ALU: AND, OR, Addition

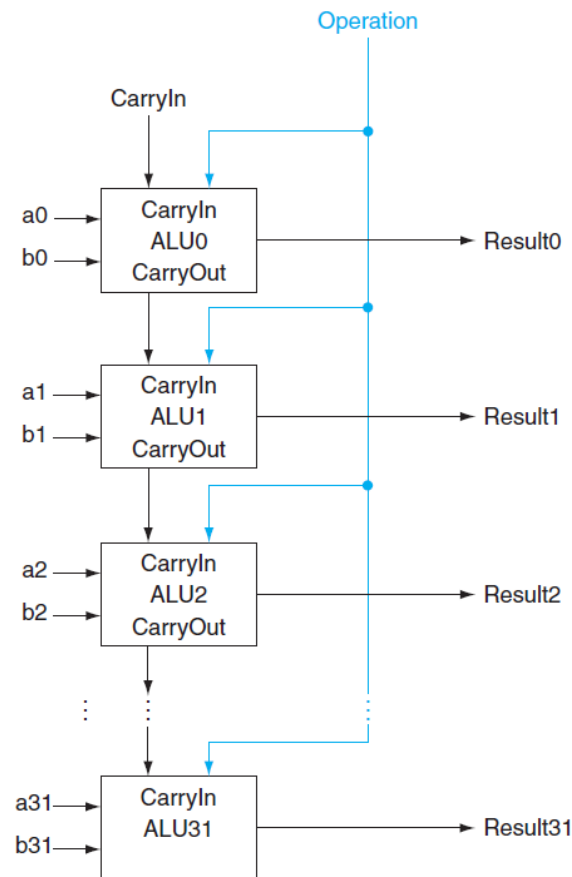
- Figure shows a 1-bit ALU derived by combining the adder with the earlier components
- Sometimes designers want ALU to perform more operations
 - To add an operation, expand multiplexor controlled by Operation line

A 1-bit ALU: AND, OR, Addition



32-Bit ALU

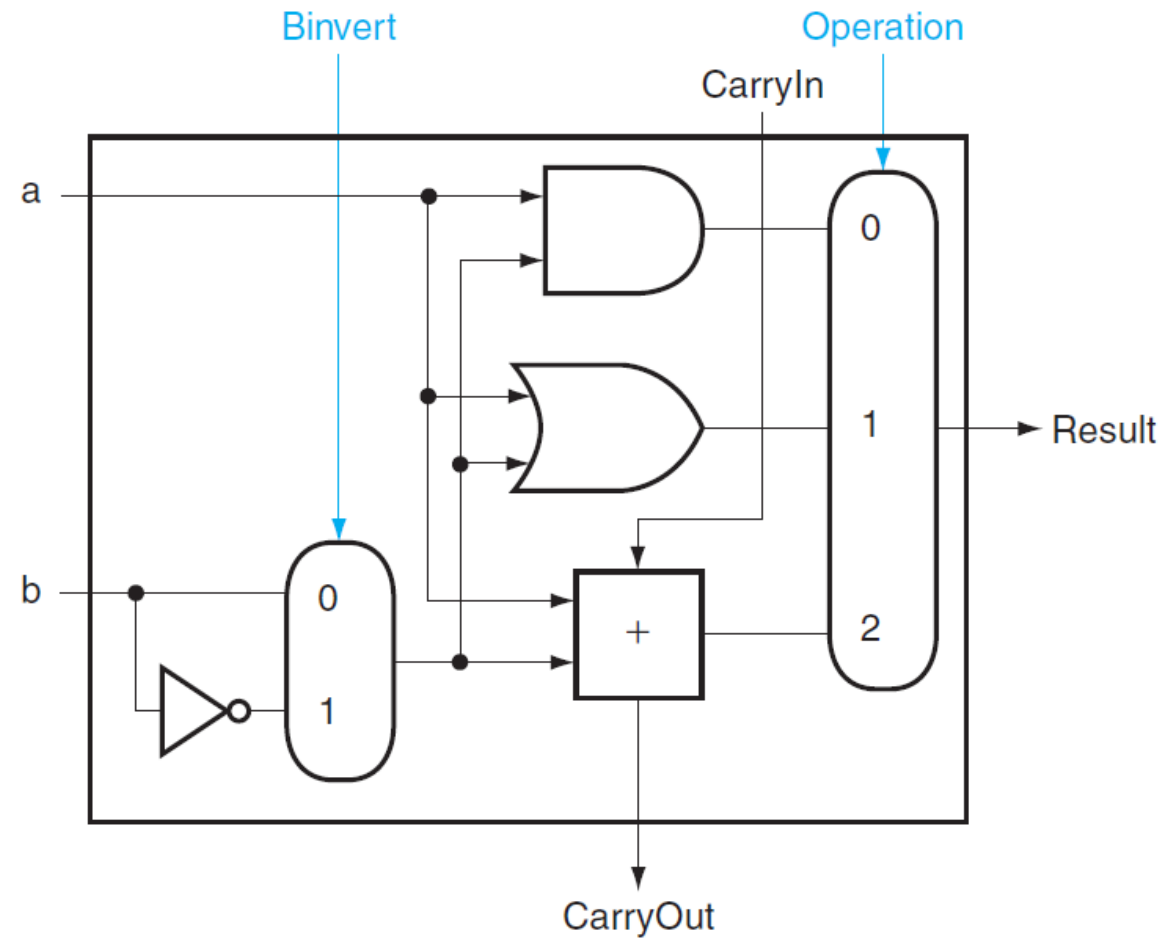
- Full 32-bit ALU is created by connecting adjacent “black boxes”



Subtraction

- Same as adding negative version of an operand
 - This is how adders perform subtraction
- To negate a number, invert each bit and then add 1
- To invert each bit, add a 2:1 multiplexor that chooses between b and b' , as shown in figure

Subtraction



Subtraction

- LSB still has a *CarryIn* signal, even though it's unnecessary for addition
- If we set this *CarryIn* to 1 instead of 0
 - Adder will calculate $a + b + 1$
 - By selecting inverted version of b , we get exactly what we want:

$$a + b' + 1 = a (b' + 1) = a + (-b) = a - b$$

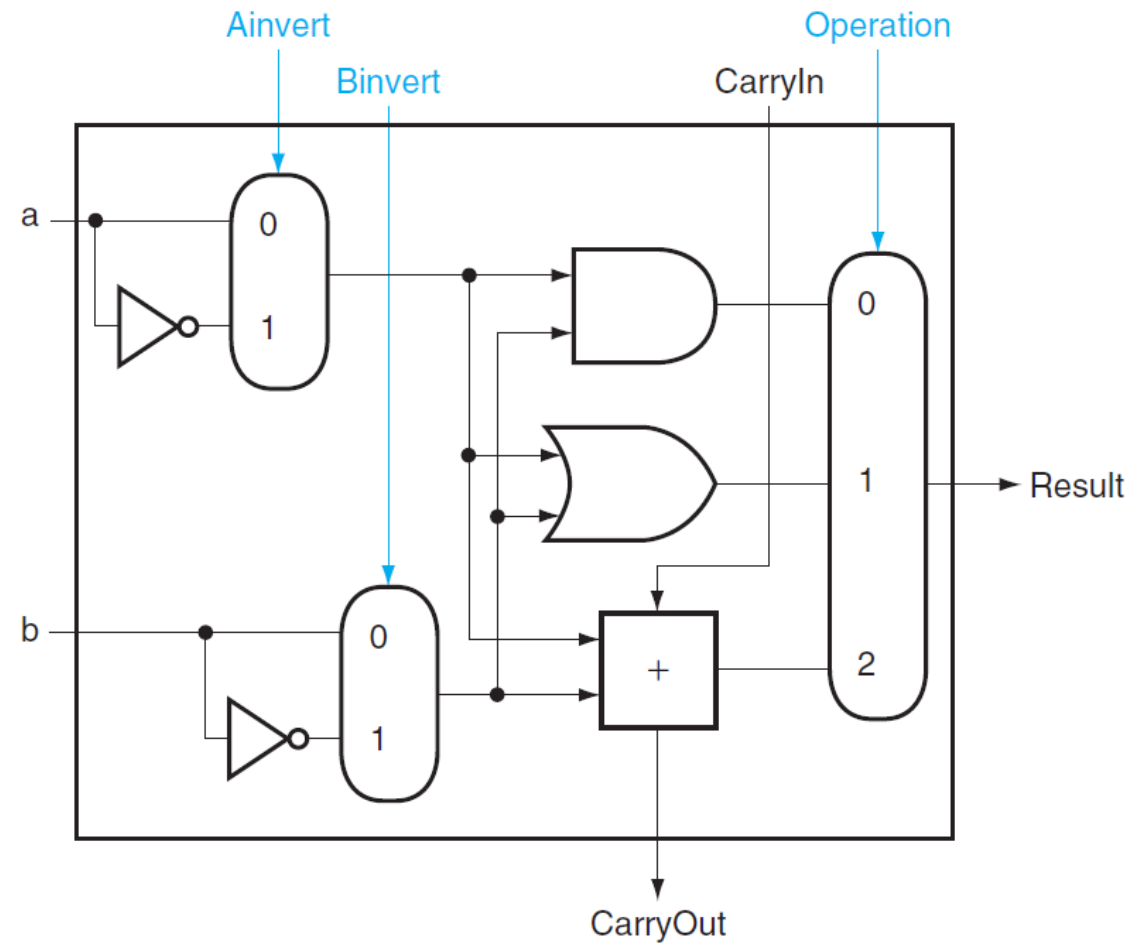
NOR

- Instead of adding a separate gate for **NOR**, we can reuse much of HW already in ALU,
- Insight comes from the following

$$(a + b)' = a' + b'$$

- Since we have **AND** and **NOT b**, only need to add **NOT a** to ALU

NOR

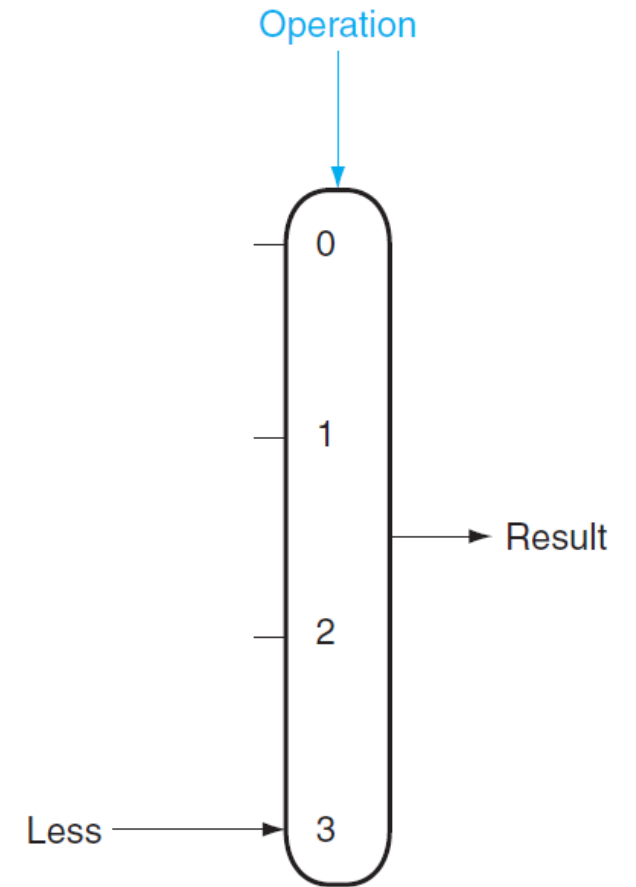


Set on Less Than

- *slt* instruction
 - Operation produces 1 if $rs < rt$, and 0 otherwise
 - Consequently, *slt* will
 - set all but LSB to 0
 - LSB set according to comparison
-
- 0: 0000 0000 0000 0000 0000 0000 0000 0000
 - 1: 0000 0000 0000 0000 0000 0000 0000 0001

Set on Less Than

- For ALU to perform *slt*
 - expand three-input multiplexor to add an input for *slt* result
 - Call that new input *Less* and use it only for *slt*
 - connect 0 to Less input for the upper 31 bits of ALU, since those bits are always set to 0



Set on Less Than

- How to compare and set LSB for *s/t* instructions?

Set on Less Than

- How to compare and set LSB for *slt* instructions?
- Subtract *b* from *a*
 - If difference is negative, then $a < b$ since

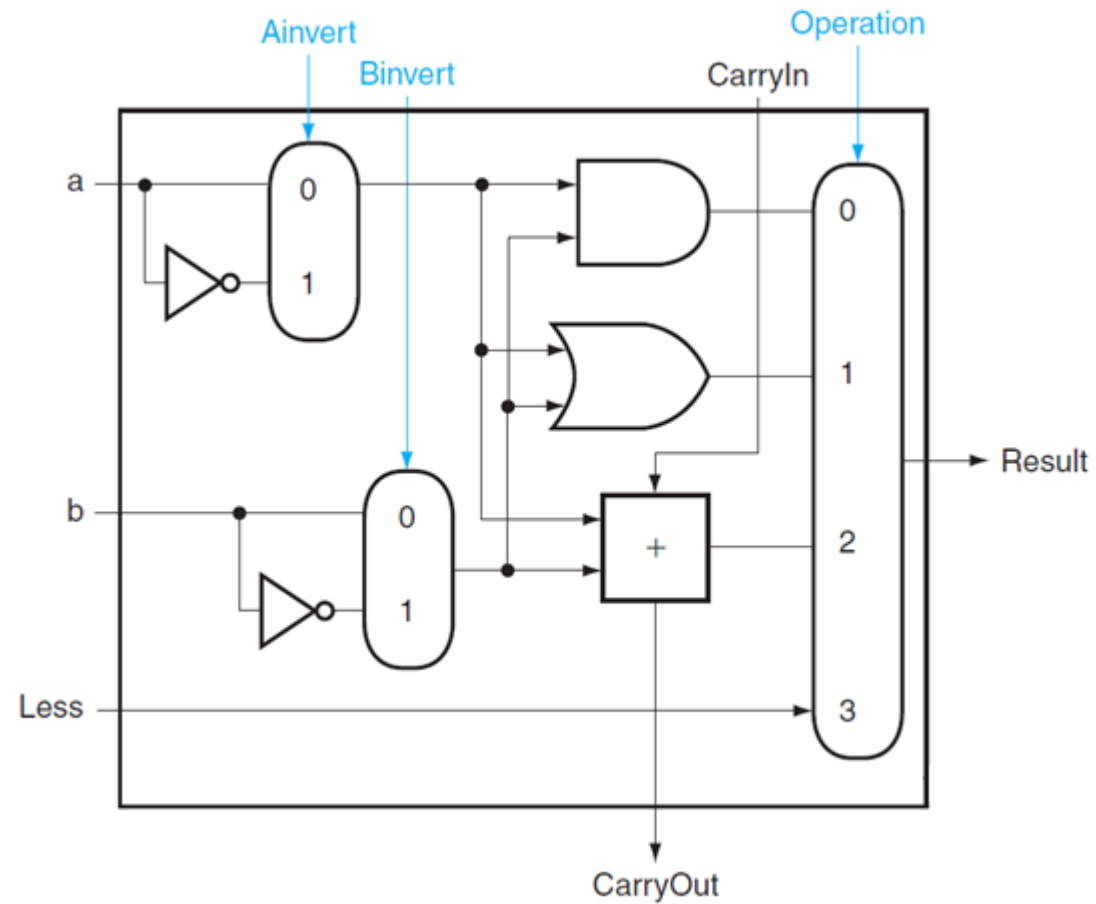
$$\begin{aligned}(a - b) < 0 &\Rightarrow ((a - b) + b) < (0 + b) \\ &\Rightarrow a < b\end{aligned}$$

- Want LSB of *slt* operation to be a 1 if $a < b$
 - i.e., 1 if $a - b$ is negative and 0 if it's positive
 - Corresponds to sign bit values: 1 means negative and 0 means positive

Set on Less Than

- Connect sign bit from adder output to LSB to get *slt*
- *Result* output from MSB ALU bit in figure for *slt* operation is *not* output of the adder
- ALU output for *slt* operation is obviously input value *Less*

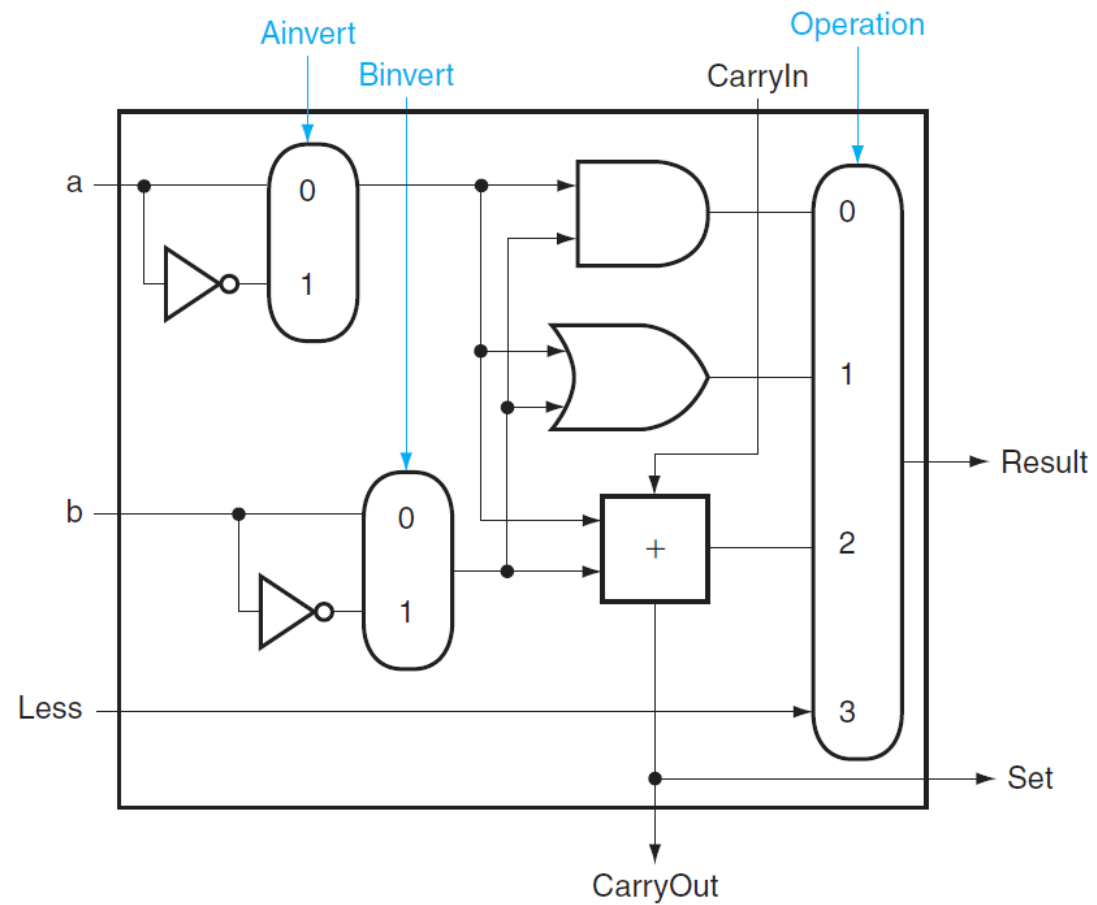
Set on Less Than



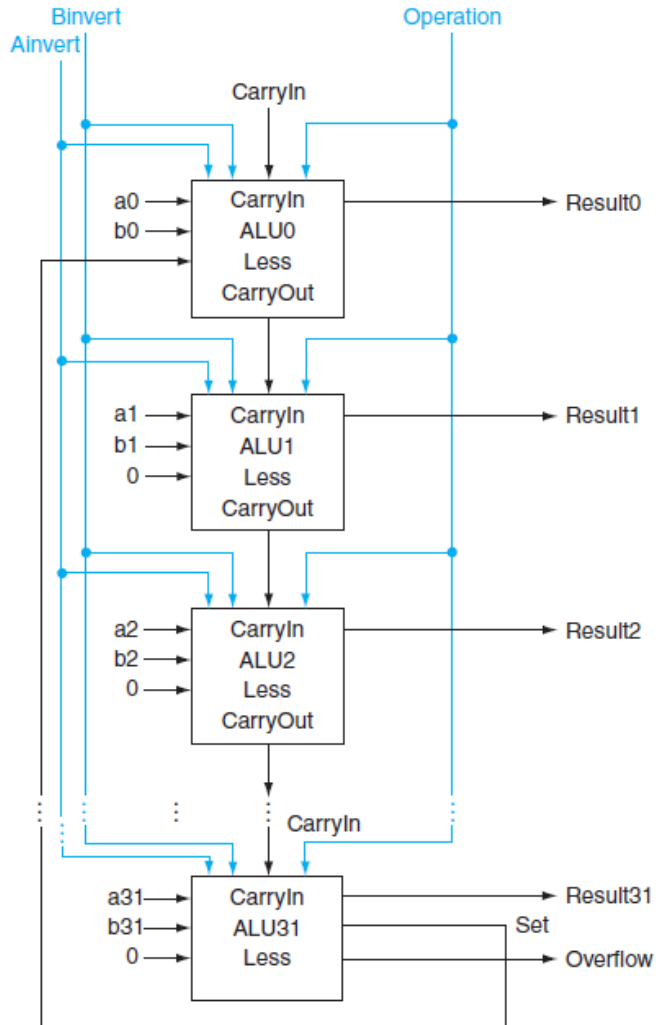
Set on Less Than

- Thus, we need a new 1-bit ALU for MSB that has an extra output bit: adder output
- Next figure shows the design, with this new adder output line called *Set*, used only for *slt*
- Notice for ALU to subtract, set both *CarryIn* and *Binvert* to 1
 - For adds or logical operations, both control lines to be 0
 - We can simplify control of ALU by combining *CarryIn* and *Binvert* to a single control line called *Bnegate*

Set on Less Than



32-bit ALU Constructed from 1-bit ALU



Branch

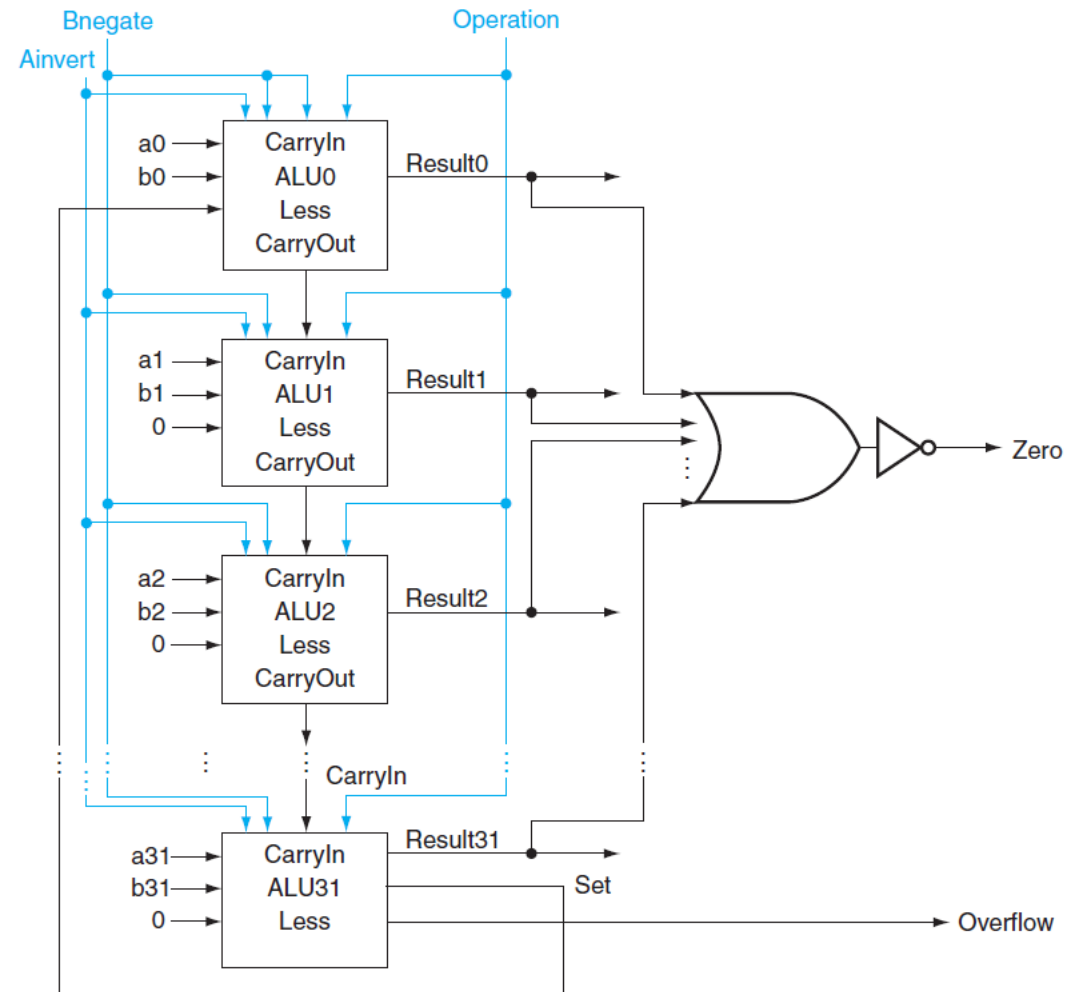
- Branch either if two registers are equal or if they are unequal
 - Way to test equality with ALU is to subtract **b** from **a**, then test to see if result is 0, since

$$(a - b = 0) \Rightarrow a = b$$

- If we add HW to test if the result is 0, we can test for equality
 - **OR** all outputs together, then send that signal through **inverter**

$$\text{Zero} = \overline{(\text{Result31} + \text{Result30} + \dots + \text{Result2} + \text{Result1} + \text{Result0})}$$

32-bit ALU



ALU Control Lines

- Can think of combination of 1-bit *Ainvert* line, 1-bit *Binvert* line, and 2-bit *Operation* lines as 4-bit control lines for ALU, telling it to perform *add*, *subtract*, *AND*, *OR*, or *set on less than*
- Figure shows ALU control lines and corresponding ALU operation

ALU Control Lines

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set on less than
1100	NOR

Universal ALU Symbol

